

Capas (OSI → TCP/IP · PDU · equipo): 7 Aplic/6 Pres/5 Sesión → Aplicación · mensaje · HTTP, DNS | 4 Transporte · segmento (TPDU) · TCP/UDP, puertos | 3 Red → Internet · paquete/datagrama · **router**, IP, ICMP | 2 Enlace (LLC 802.2 + MAC) · frame · **switch/bridge**, MAC | 1 Físico · bits · **hub**/repetidor.

Cada capa agrega header (encapsulamiento) y da servicio a la de arriba. Enlace: servicio sin conexión sin ACK / sin conexión con ACK / orientado a conexión.

1 · Teoría de la información

Info de un evento I(s) = -log₂P(s) = log₂(1/P(s)) [bits]. **Entropía** H(S) = Σ P(s) · I(s) = - Σ P(s) log₂P(s). 0 ≤ H ≤ log₂n; **máx sii equiprobable**: H=log₂n; H=0 sii determinista. Convención 0·log0=0. Interpretación: info media por símbolo = incertidumbre media.

Binaria H(p) = -p log₂p - (1-p) log₂(1-p); H(0)=H(1)=0 (sin incertidumbre), máx H(½)=1 bit (moneda justa), simétrica H(p)=H(1-p), cóncava (campana).

Memoria *imagen*: símbolos indep. ⇒ H de un bloque = suma. *Imagen* N px × k niveles equiprob ⇒ #imágenes = k^N, H(img) = N · log₂k. Tasa: R_b = H(img) · fps.

Largo medio L(C) = Σ P(s) · I(C(s)). **Sin pérdida** ⇔ H(S) ≤ L(C) (y el código es UD). **Eficiencia** η = H/L. **Óptimo**: L mínima entre todos los códigos de S. Teo. Shannon: H ≤ L_{ópt} < H+1.

Tipos: instantáneo ⇔ libre de prefijos (ninguna palabra es prefijo de otra) ⇒ **univocamente decodificable (UD)**. UD: ninguna tira admite 2 decodificaciones. **Código sufijo** (ninguna palabra es sufijo de otra) ⇒ UD (leer de der. a izq.) aunque no sea instantáneo. **No UD**: mostrar tira ambigua ⇒ **pierde info**. Kraft: instantáneo existe ⇔ Σ 2^{-l} ≤ 1. Condiciones: bloque, no singular (palabras distintas), separable (UD), instantáneo. Codificador óptimo ideal: I(s) = log₂(1/P(s)) ⇒ L=H.

Ej: S={4,3,2,...,1} H=1.842. C1(001,01,1,1010): no inst (01001), no UD ("01001"=DB=BA) ⇒ pierde info. C2(0,01,011,111): no inst, UD (sufijo) L=1.9 η= .972. C3(1,01,001,0001): inst, L=2.0 η=.923 ⇒ C2 más eficiente.

Largo fijo óptimo instantáneo: L=⌈log₂n⌉ (N=símbolos/seg). Ej 640×480 px, 10 niveles: H=307200·log₂10=1 020 496 b; fijo: |H|=1 020 497 b/img; ×30fps=30.6Mbps, 30dB ⇒ B≥30.6M/9.967≈3.07MHz.

N equiprobables (Huffman): H=log₂N. Si N=2^k: L=k=H. Si no: k=⌈log₂N⌉, m=N-2^k ⇒ **2m símbolos de k+1 bits y 2^k-m de k bits**; L=k+2m/N. N=6: H=2.585, L=2.667 · N=11.342, L=3.4.

Huffman: ordenar probs, juntar las 2 menores en un nodo (suma), repetir; 0/1 por rama. Más probable ⇒ más corto. Ej "MI MAMA ME MIMA" (15): M=1, ¯=01, A=000, I=0010, E=0011 ⇒ 33 bits (ASCII: 120).

Fuente unión D=AUB (A: n equiprob, B: m equiprob): P(a)=P_A(A)/n, P(b)=P_B(B)/m. **D equiprob ⇒ P₀(A)/n = P₀(B)/m ⇒ P₀(A)/P₀(B)=n/m**. (½, ½, 8, 16) No · (¼, ¼, 16, 16) No · (¼, ¼, 8, 16) No · (⅓, ⅓, ⅓, 12, 16) Sí (1/48).

log₂: 3→1.585 · 5→2.322 · 6→2.585 · 7→2.807 · 10→3.322 · 11→3.459 (10dB) · 101→6.658 (20dB) · 1001→9.967 (30dB) · 10001→13.288 (40dB) · 1+10¹⁰→33.22 (100dB). log₂x=ln x/ln2=3.3219·log₁₀x · 2¹⁰=1024 · 2¹⁶=65536 · 2²⁰=10⁶ · 2³²≈4.29·10⁹ · k/M/G=10³/10⁶/10⁹ en bps · 1B=8b.

2 · Nivel físico · capacidad · delay

dB: X_{dB}=10log₁₀X ⇒ X=10^{X_{dB}/10}, 3dB≈×2 · 10dB=×10 · 20=10² · 30=10³ · 40=10⁴ · 100dB=10¹⁰. **SNR en veces** en las fórmulas.

Shannon (canal c/ruido, limitado en potencia y BW): **C[bps]=B[Hz]·log₂(1+SNR)**. Sin pérdida: V_{tx} ≤ C; **salvo aclaración usar V_{tx}=C**. Despejes: **B_{min}=R_b/log₂(1+SNR)** · **SNR_{min}=2^{R_b/B}-1** (→dB) · máx fps = C/H(img).

Nyquist (sin ruido, M niveles): C=2B·log₂M. V_{tx}[bps]=V_m[bandios]·log₂M (M símbolos, N=log₂M bits/símb). Igualando Nyquist=Shannon: niveles útiles M=√(1+SNR). **Muestreo**: f_s ≥ 2f_{máx}. PCM voz: 4kHz=8000 muestras/s×8b=64kbps (125µs/muestra). T1=24×8+1=193b/125µs=1.544Mbps; E1=32×8=256b=2.048Mbps.

Ondas: T=1/f, λ=v/f, ω=2πf, s(t)=A·sen(2πft+φ). c=3·10⁸ m/s=300 000 km/s; UTP≈0.69c; fibra≈⅔c. BW: corte a -3dB (mitad de potencia). Ruido: kT=K·Tb (k Boltzmann, T absoluta); interfermodulación (m·f1±n·f2, no linealidad); diafonía (crossstalk); impulsivo (picos cortos, gran amplitud: el peor para datos). Atenuación (crece con f y distancia; ecualizar), distorsión de retardo (v depende de f, sólo guiados). **Fourier**: toda señal periódica = ½a₀+Σ[a_ncos(nωt)+b_nsen(nωt)], ω=2π/T; cuadrada = fundamental + ⅓·3^a + ⅕·5^a + ... (armónicas impares) ⇒ BW limitado deforma la señal. Medios guiados: par trenzado, coaxial, fibra (mono/multimodo, reflexión total interna), PLC, no guiados: radio, microondas, satélite, IR, láser, Li-Fi.

Modulación (dig/port. analógica): ASK (amplitud), FSK (frec), PSK (fase), QAM/ASK-PSK multinivel

(4QAM=QPSK 2b, 16QAM 4b, 64QAM 6b, 256QAM 8b; más orden = más BER). Módem: D⇒A; Códec: A⇒D. **Codif. digital**: NRZ (→0, +=1; P:der+a; C:der-A). NRZI (1=transición al inicio; falla con muchos 0). **Manchester** (bajo→alto=1, alto→bajo=0 a mitad de bit; **baud=2·bitrate**). Manchester dif. (0=transición al inicio, 1=sin transición). Multiplexación: TDM, FDM, WDM (fibra), estadística (paquetes, colas).

T_{tx}(n)=N/V_{tx} (T_{tx}(1)=1/V_{tx}) · **T_{prop}=D/V_{prop}** **Delay(1)=T_{tx}(1)+T_{prop}≈T_{prop}** **Delay(F)=T_{tx}(F)+T_{prop}** **Total: T_{prop}+T_{tx}+T_{encod}+T_{proc}: RTT=2·Delay** (simétrico). **C_{vof}[bits]=Delay·V_{tx}** (≈T_{prop}·V_{tx}, producto delay×BW) = "bits en el caño". Frames en el caño = C_{vof}/|F|. Bits en vuelo hasta que vuelve el 1er ACK = V_{tx}·RTT. **Pedido/respuesta**: T=T_{tx}(req)+T_{prop}+T_{tx}(resp)+T_{prop}. T_{prop}=50% del Delay ⇒ T_{prop}+T_{tx} ⇒ D=V_{prop}/V_{tx}.

Luna: 100Mbps, 385 000km ⇒ T_{prop}=1.2833s, RTT≈2.567s, C_{vof}≈128.3Mb; 25Mb con pedido 2kb: 20µs+1.283+0.25+1.283≈2.817s. Ej6 C_{vof}: a) C=13 288bps 5·10⁻⁴s≈6.64b · b) 1.384Mbps⇒691.9b · c) 461.3b · d) 0.461b. Ej8: 1Mbps·1.25s=1.25Mb=1250 frames de 1kb. Ej15: 640·480·8=2 457 600b/img ·20=49.15Mbps, B=4MHz ⇒ SNR=2^{12.288}-1≈5000 (37dB); T_{tx}(img)=0.05s; T_{prop}=T_{tx} ⇒ D=15 000km. Ej16: 50kb·80=4Mbps, 40Db ⇒ B≈301kHz; Delay=12.5ms+33µs; con 20kHz C=265.8kbps ⇒ H≤(70ms-33µs)·C≈18.6kb. Ej20: 5M·12=60Mb/img, C=50M·9.967=498.3Mbps ⇒ 8 img/s; Delay≈0.12s; si no es equiprob. H<60Mb ⇒ codif. óptima más corta ⇒ más img/s.

3 · Enlace punto a punto (transmisión confiable)

Caño serial (sin desorden) con ruido ⇒ framing, control de errores, ARQ (ACK+timeout). **η_{frame}=|datos|/|frame|**. Framing: **largo fijo** η=1 · **campo largo 16b** η=(16b)/L · **delimitador 8b + bit-stuffing** η=(L-8-n)/L (n=bits de stuffing; flag 01111110, se inserta 0 tras cinco 1s; si hay flag de inicio y fin, -16).

Errores: detección (paridad, CRC, checksum, MD5); corrección FEC (Hamming, Reed-Solomon). d=dist. Hamming mínima: **detectar e**: d≥2e+1 · **corregir e**: d≥2e+1. Retransmisión implícita (timeout) o explícita (NAK). **Paridad**: 1 bit, detecta #impar de errores. **Checksum Internet**: suma en complemento a 1 de palabras de 16b, se manda el complemento; receptor suma todo ⇒ 0xFFFF. **CRC** con generador (X(x) grado r: agregar r ceros al mensaje M, dividir (X(x), mod 2) por G, resto R (r bits) ⇒ se envía M||R; receptor divide M||R por G: resto=0 ⇒ error. Detecta todo error de 1 bit, 2 bits (G bien elegido), #impar si (x+1)|G, ráfagas ≤ r. CRC-16/CRC-32 (Ethernet). **Hamming (7,4)**: bits de paridad en posiciones 2^k (1,2,4); síndrome = posición del bit erróneo; d=3 ⇒ corrige 1 · detecta 2.

SWS_{ópt}=V_{tx}·RTT(F)/|F| (|F|), RTT(F)=2·Delay(F)=2(T_{tx}(F)+T_{prop}) **RWS=SWS si SACK, 1 si no** (GoBackN / S&W) **#frames a secuenciar ≥ SWS+RWS ⇒ #SEQ=⌊log₂(SWS+RWS)⌋ bits** **η_{proto}=T_{tx}(V)/RTT(F)=SWS_{tx}(F)/RTT(F)** (tope 1) Inverso con k bits de SEQ: SACK ⇒ SWS=RWS=2^{k-1}; GBN ⇒ SWS=2^k-1, RWS=1. |datos|=|F| · #SEQ · CRC(= #ACK - #SACK si piggyback). #frames=[Datos/|datos|]. Throughput útil = V_{tx}·η_{frame}·η_{proto}. η_{proto}=1 ⇒ SWS=T_{tx}(F) ≥ RTT(F).

Variables de ventana: emisor LAR (last ACK rcvd), LFS (last frame sent): LFS-LAR ≤ SWS; receptor LFR (last frame rcvd en orden), LAF (largest acceptable): LAF-LFR ≤ RWS; acepta SEQ(LFR, LAF). Con SACK se necesita SWS ≤ 2^k-1 para que la ventana nueva no se confunda con la vieja. Timeout por frame (o por ventana en GBN).

Convención cátedra: el "delay" del enunciado es T_{prop} NO Delay(F) (salvo que se diga). Delay≈T_{prop} sólo con enlaces rápidos y frames chicos.

Stop&Wait: SWS=RWS=1, SEQ de 1 bit (distinguir 2 frames, evita "reencarnaciones").

T_{total}=#frames·RTT(F). **Ventana óptima** (caño lleno): **T_{total}=#frames·|F|/V_{tx}+T_{prop}**. Ventana chica (η<1): ≈[#frames/SWS]·RTT(F).

Frames (largo fijo, CRC): S&W E: [SEQ(1b)|Datos|CRC] R:[ACK(1b)|CRC] · GBN+piggyback: único [SEQ|ACK|Datos|CRC] · SACK E:[SEQ|Datos|CRC] R:[ACK acumulativo|SACK|Padding|CRC] (padding ⇒ mismo largo). **GBN**: receptor descarta fuera de orden, emisor reenvía desde el perdido. **SACK**: receptor bufferea, ACK=próximo esperado + SACK de los recibidos; se reenvían sólo los faltantes (ej 4,5,6,7 con 4 y 6 mal: R manda ACK4-SACK5, ACK4-SACK7 ⇒ E reenvía 4 y 6 ⇒ ACK6(SACK7) ⇒ ACK8).

Ej11 (1Mbps, T_{prop}.25s, 2kb, CRC16, S&W): datos=1983b, 10 086 frames, RTT=504s ⇒ 5083s≈84.7min; 1gbps⇒84.05min; T_{prop}=1-RTT

204→34.3min. Ej12 GBN: SWS=252, RWS=1, SEQ=⌊log₂253⌋=8b, datos 1976b=10 122 fr ⇒ 20.244+25≈20.5s. Ej14 (1kb, 1Mbps, 270ms, SACK): RTT= .542 ⇒ 0.007/ .542=1.29% · 23.4% · 47.0%. Ej13: η(L)=(L-C)·log₂(4(L+D·V_{tx})/L) / L (SWS=2(L+D·V_{tx})/L, RWS=SWS); gráfico: ≤0 para L chico, crece y tiende a 1 cuando L→∞ (escalones por el |). Ej17 (2kb, 4b SEQ, SACK⇒SWS=8; 10kbps, 1s): 8 · 2/2.4=66.7%. Ej18: SEQ 10b + SACK ⇒ SWS=RWS=512; 2 distancia: η_{frame} igual, η_{proto} baja (SWS fijo); c) datos=2000-46=1954b ⇒ 10 236 fr; RTT=512·2000/1M=1.024s ⇒ T_{prop}≈.51s ⇒ T≈20.47+ .51≈21s. Ej19: SACK16b+SACK ⇒ SWS=RWS=2¹⁵; frame [SEQ16|Datos|CRC16] η=992/1024=.969; 6min: η=2¹⁵·17.07µs/720s≈0.078%. Parcial (10Mbps, SACK, 5kb, SEQ8b ⇒ SWS=128): 100% ⇔ 128·0.5ms ≥ RTT ⇒ Delay(F)=32ms (T_{prop}=31.5ms); delay 1s ⇒ SWS=4002 ⇒ 13 bits.

4 · Medios compartidos · Ethernet · Switching

Delay en red switchheada: D=T_{tx}+T_{prop}+T_{queue} (tiempo encolado en switches; store&forward: T_{tx} en cada salto).

Preámb	Dest 6B	Src 6B	Type/Le n 2B	Datos 46- 1500B	CRC 4B
78+ufld 1B (64b)					

Frame 802.3: min 64B=512 bits (sin preámbulo: 6+6+2+46+4), máx 1518B, MTU=1500. Datos <46B ⇒ **padding**; el receptor lo descarta usando el campo **Length** (802.3) / info de capa sup. Dir. MAC 48b; broadcast FF:FF:FF:FF:FF:FF. Ethernet clásica 10Mbps (CSMA/CD) vs conmutada (switches, full-duplex, sin colisiones).

CSMA/CD 1-persistente: sensa; libre ⇒ transmite; ocupado ⇒ espera y transmite apenas se libera (⇒ 2 que esperaban **colisionan**). Para detectar colisión hay que **estar transmitiendo**. Peor caso: A transmite, B justo antes de que le llegue la señal sensa libre y transmite; la colisión vuelve a A ⇒ **t_{min}=2τ** (τ=delay máx punta a punta). **|F_{min}=V_{tx}·2τ** (10Mbps=51.2µs=512b). Colisión ⇒ **jam + backoff exponencial**: tras n-ésima colisión espera K·slot(51.2µs), K∈[0, 2^{min(n,10)-1}]; aborta a las 16. Un host lejano puede sensar libre porque la señal aún no llegó (τ) ⇒ colisión.

Distancia máx de un dominio de colisión: D_{máx}=V_{prop}/|F_{min}|/(2V_{tx}) (↑ V_{tx} con mismo 512b ⇒ ↓ diámetro). Variantes CSMA: **1-persistente** (espera y transmite ya: colisiones al liberarse), **no persistente** (si ocupado espera tiempo aleatorio y re-sensa), **p-persistente** (ranurado; transmite con prob. p). Estados transmisor CSMA/CD: sensar ⇒ transmitir (o colisión) jam ⇒ backoff ⇒ reintentar. **ALOHA** puro S=Ge^{-2G} máx 18% (G=.5); ranurado Ge^{-G} máx 37% (G=1). S=G(1-P_{co}). CSMA escala mal con G. **Wireless CSMA/CA**: sensa, espera IFS, backoff aleatorio en ventana de contención (slots, se congela si se ocupa), ACK; sin ACK ⇒ asume colisión y retransmite. **Estación oculta** (A→B, C no oye a A, transmite e interfiere en B) / **expuesta** (C oye B→A y no manda a D aunque podría).

Domínios: Hub (L1) = un solo dominio de colisión. Bridge/Switch (L2) separa dominios de colisión, **mismo dominio de broadcast**. Router (L3) separa broadcast (y dominios de STP). VLAN (802.1q, trunks): particiona lógicamente la LAN.

Learning bridge: aprende **MAC ORIGEN** ⇒ **puerto de entrada**. Destino conocido ⇒ sólo por ese puerto (si es el de entrada, filtra). Desconocido ⇒ **flood** (todos menos el de entrada). Broadcast ⇒ siempre a todos. Tabla = optimización (no necesita estar completa, entradas expiran). **Flooding ⇔ broadcast**: flooding reenvía sin cambiar direcciones; broadcast es una dirección que todos aceptan y procesan. Con ciclos ⇒ tormentas ⇒ STP.

STP. BPDU=(rootID, distancia, switchID) ("soy switchID, creo que el root es rootID, estoy a distancia d"). **b1 mejor que b2** ⇒ rootID menor; empate ⇒ distancia menor; empate ⇒ switchID menor (luego puerto menor). 1) **Root**=menor ID; **todos sus puertos DP**. 2) **Root Port** (1 por switch no root): donde ve el mejor BPDU al root (menor costo; empate: menor ID del vecino; luego menor puerto). 3) **Designated Port** (1 por segmento): el switch que ofrece el mejor BPDU en ese segmento (menor dist.; empate menor ID). Puertos a hosts/segmentos stub ⇒ DP. 4) **Blocked/Closed**: el resto (no reenvía datos, si escucha BPDUs). Costo 1 por segmento. #bloqueados (enlaces switch-switch p2p) = E-(V-1). Tras STP, learning/flood sólo por RP/DP. Si se cae un enlace se recalcula: puertos bloqueados pueden pasar a DP. Agregar enlace ⇒ nuevo ciclo ⇒ se bloquea uno. Router corta STP ⇒ un árbol por dominio L2.

Ej3 (L1{S1,S2}, L2{S1,S3}, L3{S2,S3}, L4{S3}): root S1; S2 RP→L1, DP→L3 (1,1,2)<(1,1,3); S3 RP→L2, **BLOCK→L3**, DP→L4. Se corta S2-L1: S2 RP→L3 (dist 2), S3→L3 pasa a DP, sin bloqueados. Ej4 (clase; 1001-1002,1001-1005,1002-1003,1005-1004,1005-1003,1003-1004,1004-1006): root 1001; 1002 RP→1001; 1005 RP→1001; 1003 d=2 RP→1002 (empate, menor ID), **BLOCK→1005**; 1004 d=2 RP→1005, **BLOCK→1003** (1003 DP ahí, menor ID); 1006 RP→1004. Ej7 K: root=menor ID (S1); S2..S5 RP al enlace directo con S1; en los 6 enlaces restantes DP el de menor ID, bloquea el mayor (S5,3, S4,2, S3:1 ⇒ 6 = 10-4). Ej5: si X aprendió H1→0 y H2→2, sus puertos 0 y 2 reenvían ⇒ elegir IDs para que el ciclo se bloquee del lado del puerto 1 (root del lado del 2, vecino por 1 con ID < X). Minimizar delay A→B (Ej6/8): el tráfico sigue el árbol ⇒ delay A≈#segmentos del camino en el árbol ×(T_{tx}+T_{prop}+T_q); poner root en el camino/entre ellos (ID) o recabarlar para que el camino del árbol sea el directo. Ej9: un **broadcast** (ARP who-has) recorre todo el árbol ⇒ **todos** los switches aprenden la MAC origen; el reply unicast sólo enseña a los del camino; luego una trama a MAC conocida va sólo por el camino del árbol. Colisiones repetidas en 5 RTT: posible (reintentos con K=0), poco esperable (backoff aleatorio creciente). Ej2 learning: X→W flood (todos aprenden X, Y lo ve); Z→X directo (aprenden Z, Y no lo ve); Y→X (B2,B1 aprenden Y; Z no); W→Y: B3 no conoce Y ⇒ flood (Z lo ve), B2 lo entrega.

△ **Trampas carilla 1**: "delay" dado = T_{prop}. RTT(F)=2(T_{tx}(F)+T_{prop}), no 2T_{prop} · SWS y #frames con | · #SEQ usa SWS+RWS · |datos| descuenta SEQ, ACK, SACK, CRC · SNR en dB ⇒ veces antes de log · kbps=10³ (no 1024); B=byte, b=bit · H de una imagen = #px·log₂(niveles) (no log de #px) · largo fijo ⇒ |H| · τ de CSMA/CD es de ida (espera 2τ) · learning bridge aprende del **origen** · STP: desempatar por ID del emisor; en cada segmento hay 1 solo DP · puerto hacia host/segmento stub · DP · switch no cambia direcciones al floodear.

5 · Nivel de red · IP

	Circuito virtual (X.25, FR, ATM)	Datagrama (IP)
Setup	si (≈1 RTT; reserva recursos)	no, manda cuando quiere
Hdr	VCI corto (dir. completa sólo en setup)	dir. destino completa (más overhead)
Tabla	(puerto _n , VCI _n)→(puert _{Out} , VCI _{Out}), estado por conexión	red destino→próx. salto
Falla	se caen todos los VC que pasan; re-establecer	sólo se pierden paquetes en vuelo; reruteo
Orden	un camino ⇒ en orden	puede desordenar/duplicar/p order

Circuitos virtuales: PVC (lo arma/baja el admin) o SVC (mensaje de setup/fin salto a fin; cada switch agrega/borra su fila, paquetes tras el fin se descartan). Label switching: el VCI cambia en cada salto.

Contar conexiones VC: encadenar filas (out de un switch = in del siguiente) hasta un host; cada cadena = 1 conexión (unidireccional).

Ver 4	HLen 4	TOS/DiffServ-ECN 8	Total Length 16 (bytes, ≤65535)	
Ident 16			Flags 3 (0,DF,MF)	Frag Offset 13 (×8B)
TTL 8		Protocol 8		Header Checksum 16
Source Address 32				
Destination Address 32				
Options (var) + Pad → Data				

IPv4 best effort, sin conexión. HLen en palabras de 32b (5-15 ⇒ header 20-60B). Protocol: 1 ICMP, 2 IGMP, 6 TCP, 17 UDP, 89 OSPF. Checksum sólo del header. **Versión va primero**: el formato del resto depende de ella (v6 distinto). **Router modifica**: TTL (-1, 0 ⇒ descarta + ICMP time exceeded), checksum (recalcula), y al fragmentar Total

Length/Flags/Offset (y options). Máx 65535B pero no se usa: MTU de enlace (Ethernet 1500, FDDI 4500) ⇒ fragmentación.

Fragmentación: si MTU < datagrama. Fragmentos = datagramas autocontenidos, mismo **Ident**, heredan header; **MF=1** en todos menos el último; **Offset** en unidades de 8B; datos por frag = 1 (MTU-HLen)/8 | 8; reensamblado sólo en destino; IP no recupera fragmentos perdidos; DF=1 ⇒ descarta+ICMP. MTU mínimo 68B (60+8). Ej: 4000B (3980 datos) MTU1500 ⇒ 1480/1480/1020, offsets 0/185/370, MF 1/1/0.

Machete 2/2: Red (IP · Forwarding · Ruteo) · Transporte (TCP/UDP)

5 · IP (cont.) · direccionamiento · forwarding

/bits	/1	/2	/3	/4	/5	/6	/7	/8
128	192	224	240	248	252	254	255	
bloq.	128	64	32	16	8	4	2	1

/n: n bits de red (1s contiguos; 255.255.0.255 inválida), h=32-n de host. #**direccion=2ⁿ**, **hosts=2^{h-2}** (red y broadcast). **Red**=bits de host en 0 (IP AND máscara); **broadcast**=bits de host en 1; hosts: red+1 ... bcast-1. Bloque en el octeto de corte = 256-valor máscara; red = [x/bloque] ·bloque. /24:254 · /25:126 · /26:62 · /27:30 (0-31) · /28:14 · /29:6 · /30:2 (enlaces p2p) · /22:1022 · /23:510 · /16:65534.
Ej 135.46.56.0/22: 3er oct 56=00111000, host bits→1: 59 → bcast 135.46.59.255, hosts 56.1→59.254. 147.156.249.228/22 → red 147.156.248.0, bcast .251.255.

Subnetting / VLSM: ordenar subredes por #hosts desc; para cada una h=[log:(hosts+2)] (contar interfaz del router!), prefijo 32-h; asignar bloques alineados (dirección de red múltiplo del tamaño del bloque) sin solaparse; enlaces router-router /30. **Agregación / supernetting** (resumir rutas): prefijo común más largo, ej 192.53.40.0/24+192.53.41.0/24 = 192.53.40.0/23. Una red "X/n" válida sólo si X tiene los h bits de host en 0.
Especiales: privadas RFC1918 **10/8, 172.16/12, 192.168/16** · loopback 127/8 (no se rutea) · link-local 169.254/16 · 0.0.0.0/0 default · 255.255.255.255 bcast limitado · bcast dirigido = bcast de una red remota (se rutea como esa red). Classful: A 0-127 /8, B 128-191 /16, C 192-223 /24, D 224-239 multicast, E. IPv6 128 bits (8 grupos hex de 16b, 8000:123:4567:89AB:CDEF), header fijo 40B, sin checksum, sin fragmentar en routers, Hop Limit, Next Header, Flow Label. **NAT:** traduce IP privada:puerto ↔ IP pública:puerto. **DHCP** (UDP 67/68): Discover (bcast) → Offer → Request → Ack; da IP, máscara, GW, DNS, lease.

Forwarding: tabla (Red destino, Próximo salto); próx. **salto = interfaz** si la red está directamente conectada, **IP del próximo router** si es remota (esa IP debe estar en una red directa → re-lookup para saber la interfaz). **Longest prefix match:** de las entradas con (IP AND máscara)=red, gana la más específica; si ninguna → **Default**; sin default → **descarta** (·HICMP destino unreachable; ¡un router no floodea!). La IP destino no cambia en el camino; la MAC sí en cada salto.
Host: destino en mi red → directo; si no → default gateway (debe estar en su misma subred). Algoritmo: ¿red de alguna interfaz? directo · ¿en tabla? next hop · default.
ARP (RFC 826): IP→MAC en la LAN. Request **broadcast** "who-has X", reply **unicast**, caché con TTL, gratuitous ARP (anuncio). **Primero ruteo (¿por dónde?), después ARP (¿a qué MAC?)**; se hace ARP del next hop, no del destino remoto. En tabla ARP, Age "-" = interfaz propia.

Armar direccionamiento (EJ5): una subred por segmento L2 (switch/LAN) y por enlace router-router; routers .1 (o 1ª IP), PCs siguientes; gateway de cada PC = IP del router en SU red. Tablas router: redes directas→interfaz, remotas→IP del router vecino (en red compartida). PCs: red local directa + default→router; si hay 2 routers en su red, rutas explícitas o default a uno.
Debug de redes (ping falla): chequear máscara igual en host, interfaz del router y tabla · gateway dentro de la subred del host · ruta de ida y de vuelta (el reply puede perderse) · defaults mutuos R1→R2 → **loop hasta TTL=0** (sin default → descarte limpio) · IP de red/bcast asignada a host · subredes solapadas. Ej6: tablas y IF con /26 (0-63) pero red real /25 → reply a .90 cae en defaults → loop. **ipconfig:** Physical=MAC (fábrica, L2), IP (L3), Mask (tamaño subred), Gateway (router para destinos fuera de la subred). 2 interfaces en la misma red lógica = mismo gateway. ICMP (sobre IP, proto 1): tipo 8 echo request / 0 echo reply (ping, mide RTT), 11 time exceeded (TTL=0; traceroute manda TTL=1,2,3... y cada router del camino responde), 3 dest unreachable (red/host/puerto; "fragmentation needed" si DF), 5 redirect. DHCP asigna IP/máscara/GW/DNS.

Paquete salto a salto (H1→R1→R2→H2): IP src/dst fijas (H1,H2) todo el camino; en cada LAN la trama lleva MAC src = interfaz saliente, MAC dst = next hop (R1, luego R2, luego H2), resuelta con ARP; TTL-1 y checksum nuevo en cada router.
Ej4 A: 57.14→56/22 IF1 · 63.10→60/22 IF0 · 52.2, 208.70.188.15→Default 62.100 · 62.62→IF0 · 192.53.40.7 matchea /23 y /24 → /24 → 60.100

(≤60/22 → sale IF0) · 56.7→IF1. B (sin default): sólo 192.53.40.7→IF1 y 56.7→IF0 (≤56.0/25); resto **descarta**. Ej14: 127.0.0.1 no se reenvía; 192.168.1.1 privada igual va a default (filtrar es de firewall).

6 · Ruteo

Forwarding (elegir puerto de salida con la tabla) vs **Routing** (construir la tabla). Routing table (lógica, cómo se construyó) → forwarding table (red, next hop, MAC...). **Estático** (manual, sin overhead, no reacciona a fallas, no escala) vs **dinámico** (protocolo, se adapta, consume red/CPU). **IGP intradominio** (dentro de un AS): RIP (DV), OSPF (LS). **EGP interdominio:** BGP (path vector: publica rutas como secuencias de AS; AS stub/multihomed(sin tránsito)/transit; portavoz BGP). Ruteo = camino de menor costo en grafo pesado. BGP evita loops descartando rutas cuyo AS-path ya contiene su propio AS; política > costo; corre sobre TCP (179). NAPs/IXPs (CABASE) intercambian tráfico entre AS.

Distance Vector (RIP): cada nodo tiene (Destino, Costo, NextHop); manda su **vector completo (Destino,Costo)** sólo a **vecinos directos, periódico (30s)** + triggered al cambiar. **Bellman-Ford:** D_x(y)=min_v{c(x,v)+D_v(y)}; al recibir: costo=recibido+1, se aprende si es menor o si viene del next hop actual (refresh; timeout borra). Métrica RIP = saltos (costo 1), válidas 1-15, **16=∞** → redes chicas. UDP 520. Boot: 0 a sí, 1 a vecinos, ∞ resto; tras k intercambios conoce hasta k+1 saltos; converge en ≈diámetro rondas. Mensaje RIP = pares (RED/máscara, COSTO). **Conteo a infinito** (cae enlace, un vecino anuncia ruta vieja que pasaba por mí): mitigaciones: ∞=16, **split horizon** (no anunciar la ruta al vecino que me la enseñó), **poison reverse** (anunciarla con ∞) — sólo resuelven ciclos de 2 nodos.

Link State (OSPF): cada nodo manda **sólo el estado de sus enlaces directos a toda la red** (flooding confiable). **LSP**=(ID nodo, SeqNum, TTL, lista (vecino/red, costo)). Cada router: guarda el LSP más reciente de cada nodo (**mayor SEQ gana**), decrementa TTL (0 → descarta), reenvía a todos menos a quien se lo mandó, ACK; genera LSP periódico/ante cambios con SEQ+1 (SEQ=0 al reiniciar). ¿SEQ y TTL? SEQ: distinguir nuevo de viejo/duplicado; TTL: que LSPs viejos mueran. Cada nodo arma el mapa completo y corre **Dijkstra (forward search):** Confirmed={ (yo,0,-) }; tomar último confirmado, leer su LSP, por cada vecino costo=c(next)+c(next,vec): si no está → Tentative; si está con mayor costo → reemplazar (y NextHop); mover el menor de Tentative a Confirmed; repetir hasta vaciar. **Costo OSPF=10¹⁰/BW[bps]** (10M→1000, 100M→100, 1G→10). OSPF: sobre IP (proto 89), autenticado, ECMP, varias métricas por ToS, **áreas** (truncal, routers de frontera de área resumen; escala, pierde optimalidad). Mensajes: Hello (vecinos), Database Description, LS Request, LS Update, LS ACK (confirmados). LSPs contradictorios A-B: diferencias de tiempo en el flooding; usar SEQ/age (no timestamps); enlace válido sólo si ambos extremos lo reportan (two-way) → asumir caído.

	DV / RIP	LS / OSPF
Qué manda	toda su tabla (dest,costo)	sólo sus enlaces directos
A quién	vecinos	toda la red (inundación)
Algoritmo	Bellman-Ford distribuido	Dijkstra local
Memoria	su vector O(N)	topología completa (1 LSP/nodo)
CPU	baja (min local)	alta (SPF por cambio)
Converg.	lenta, ciclos, conteo a ∞	rápida, estable, visión consistente
Escala	mal (≤15 saltos)	mejor (áreas)

Routing table vs forwarding table (EJ17): routing = red, máscara, costo/métrica, next hop, cómo se aprendió (C conectada, R RIP, O OSPF); forwarding = red→next hop/interfaz (+MAC). **Mensaje OSPF de R:** (ID=R, SEQ, TTL, (red o vecino, costo) por cada enlace directo)) (igual salvo SEQ/TTL en cada iteración). **RIP ante caída de una red (EJ16):** antes: anuncia (red, d); después: el router directo anuncia (red, 16) (poison) → vecinos pasan a 16 y luego la borran; con split horizon no se la devuelven al que la enseñó.

¿Cambio más corto Vt? Flooding: sí (explora todos, ineficiente). OSPF: sí sólo convergido. RIP: sólo si mín saltos = mín costo y convergido. Estático: sólo si la tabla refleja la realidad.

Overhead de ruteo (EJ13), update cada 30s, header 32b, dir 32b, métrica 32b: **RIP:** msj=32+64-R

(R=#destinos), #msj/ciclo=2·E→ bps=2E(32+64R)/30. **OSPF:** LSA=32+64-grado; cada LSA cruza cada enlace = bps=£LSA/30 (con duplicados reales del flooding es más). Árboles ≈ igual; malla densa OSPF ≫ RIP (K5: 234.7 vs 480bps).

Ej15 (R con 10.0.2.1/24, 10.0.3.1/30; recibe RIP 10.0.2.0:1, 10.0.3.0:0, 10.0.4.0:0, 10.0.5.0:1): vecino R2=10.0.3.2 tiene directas 10.0.3.0/30 y 10.0.4.0/24; 10.0.5.0 detrás de un R3. Tabla R1: 10.0.2.0/24→IF0/0 · 10.0.3.0/30→IF0/1 · 10.0.4.0/24 y 10.0.5.0/24→10.0.3.2. Ej12 OSPF: A recibe LSP B(A5,C2,D2), C(A2,B2,D1), D(B2,C1) (el de D con TTL-1); Dijkstra: C(2,C), D(3,C), B(4,C). Ej7: A→D directo 8 pierde contra A→C→E→D=6.

7 · Transporte · TCP · UDP

Puerto: 16 bits, identifica el proceso/socket → **(de)multiplexación.** Sin puertos: no se sabe a qué proceso entregar. Conexión TCP = **4-tupla** (IPsrc, IPdst, IPdst, Pdst). Server escucha en puerto fijo (80 HTTP, 443 HTTPS, 53 DNS, 22 SSH, 23 Telnet, 25 SMTP, 520 RIP); cliente usa efímero. **UDP:** sin conexión, no confiable, sin orden/flujo/congestión, datagramas, header 8B (Psrc, Pdst, Length, Checksum) = tiempo real (RTP), DNS, juegos. **TCP:** orientado a conexión, confiable (ACK, checksum, SEQ, retransmisión por timeout), en orden, flujo de bytes, full-duplex (2 flujos), control de flujo y de congestión → web, archivos, mail.

Enlace vs transporte (mismas herramientas: SEQ, ACK, checksum, ARQ, ventana): transporte conecta muchas máquinas → setup/release explícitos; RTT muy variable → **timeouts adaptativos**; paquetes muy viejos → espacio SEQ grande (32b); receptores distintos → ventana anunciada; red compartida → congestión. Sobre red VC en orden sin control de errores: no se divide por varios caminos; full-duplex con 2 VC simplex; SEQ sí (detectar pérdidas), no para ordenar.

SrcPort 16	DstPort 16
SequenceNum 32 (1er byte del segmento)	
Acknowledgment 32 (próximo byte esperado)	
HdrLen 4	0 · URG ACK PSH RST SYN FIN AdvertisedWindow 16
Checksum 16 (+pseudo header IP)	UrgPtr 16
Options (MSS, window scale, SACK, timestamps) → Data	

SEQ/ACK: SEQ=nº del 1er byte; **ACK=próximo byte esperado** (acumulativo). **SYN y FIN consumen 1 nº:** next SEQ=SEQ+en(+1 si SYN/FIN). ISN aleatorio. ACK flag en todo segmento salvo el 1er SYN. Segmentos sin datos (ACK puro) no avanzan SEQ.

3-way: C→S SYN(seq=x) · S→C SYN+ACK(seq=y, ack=x+1) · C→S ACK(seq=x+1, ack=y+1) (puede llevar datos). **Cierre 4-way:** FIN→ACK, FIN→ACK (ACK+FIN juntos → 3 segmentos). **RST:** aborta conexión "confusa" (SYN en ESTABLISHED, ACK fuera de ventana = posible inyección/ataque, segmento viejo, host reiniciado); respuesta de puerto cerrado a un SYN (RST+ACK). Recibir RST → CLOSED (el otro lado puede seguir ESTABLISHED si no se enteró).

Máquina de estados (evento/acción): CLOSED → listen → **LISTEN** · CLOSED → connect/SYN → **SYN_SENT** · LISTEN → SYN/SYN+ACK → **SYN_RCVD** · ACK → **ESTABLISHED** · SYN_SENT → SYN+ACK/ACK → ESTABLISHED · SYN_SENT → SYN/SYN+ACK → SYN_RCVD (open simultáneo) · SYN_RCVD → RST → LISTEN
Cierre activo: ESTAB → close/FIN → **FIN_WAIT_1** → ACK → **FIN_WAIT_2** → FIN/ACK → **TIME_WAIT** → FIN/ACK → **CLOSING** · ACK → TIME_WAIT (**cierre simultáneo:** ambos FIN antes de recibir el otro) · FIN+ACK/ACK → TIME_WAIT directo.
Cierre pasivo: ESTAB → FIN/ACK → **CLOSE_WAIT** → close/FIN → **LAST_ACK** · ACK → CLOSED.
TIME_WAIT (lo transita quien cierra primero / manda el último ACK) -timeout **2·MSL** → CLOSED (MSL típ. 2 min RFC793 → 4 min). SYN_RCVD → close/FIN → FIN_WAIT_1.
Cliente: CLOSED→SYN_SENT→ESTAB (al mandar el 3er ACK). Server: LISTEN→SYN_RCVD→ESTAB (al recibir el 3er ACK). netstat: LISTEN 0.0.0.0:p*.*.

Ventana/flujo: TCP = sliding window por bytes (GBN-like, ventana anunciada por el receptor). Rx: AdvWin=MaxRcvBuffer · (LastByteRcvd-LastByteRead). Tx: LastByteSent-LastByteAcked ≤ AdvWin; EffWin=AdvWin-(LastByteSent-LastByteAcked). AdvWin=0 → **persist timer** (probes de 1B).

Throughput ≤ Ventana/RTT; caño lleno → Ventana ≥ BW·RTT (AdvWin 16b = 64Kb; window scale RFC1323). Segmento sale si: lleno (MSS), timeout o PUSH. MSS def. 536B (MTU def 576); Ethernet: 1500-20-20-1460.

RTT/RTO. Muestra = segmento que consume SEQ (datos/SYN/FIN) ↔ el ACK que lo cubre. **Karn/Partridge: no usar segmentos retransmitidos** (ACK ambiguo) y duplicar TO tras cada retransmisión (backoff exp). Original: EstRTT=α·EstRTT+(1-α)·Sample (α∈[8,.9]), TO=2·EstRTT. **Jacobson/Karels:** Diff=Sample-Est; Est+δ·Diff; Dev+δ(|Diff| - Dev) (δ≈1/8). **RTO=μ·Est+φ·Dev** (μ=1, φ=4). **6c298:** RTO inicial 1s; 1ª medida R: SRTT=R, RTTVAR=R/2; luego RTTVAR=(1-β)RTTVAR+βSRTT-R' (β=¼, con SRTT viejo), SRTT=(1-α)SRTT+αR' (α=½); RTO=SRTT+max(G, 4·RTTVAR), mín 1s.

Control de congestión (Tx): EffWin=min(cwnd, AdvWin)-en vuelo. **Slow start:** cwnd=1 MSS, +1 MSS por ACK (→ ×2 por RTT) hasta ssthresh. **Congestion avoidance (AIMD):** +1 MSS por RTT (+MSS/cwnd por ACK). **Timeout** → ssthresh=cwnd/2, cwnd=1 MSS, slow start. **3 ACKs duplicados = fast retransmit** (reenvía sin esperar TO) + **fast recovery** (Reno: ssthresh=cwnd/2, cwnd=ssthresh, sigue en AIMD). Pérdida → multiplicative decrease.

Sockets (Berkeley): server socket→bind→listen (LISTEN)→accept (bloquea hasta ESTAB); cliente socket→connect (manda SYN)→send/recv→close (manda FIN). UDP: sendto/recvfrom sin conexión. **netstat -an** muestra estados; nc-4 5555 → LISTEN; nc localhost 5555 → 2 filas ESTABLISHED (cliente y server); cerrar → quien cierra primero queda TIME_WAIT.

Analizar trazas: 1) agrupar por 4-tupla (sin importar sentido) → #conexiones; 2) cliente = quien manda el SYN sin ACK; 3) seguir SEQ/ACK: ack esperado = seq+en(+1 SYN/FIN) del otro; 4) mismo seq+en repetido → retransmisión; 5) flags ilegales o ack fuera de ventana → anomalía = RST; 6) marcar estados con cada envío/recepción. Header TCP 20-60B (HdrLen en palabras de 32b). Checksum TCP/UDP cubre pseudo-header IP (src, dst, protocolo, length) + header + datos. Overhead mínimo por segmento sobre Ethernet: 20 TCP+20 IP+18 Eth(+8 preámbulo).

Nagle: si hay ≥MSS (y ventana) manda full; si no, y hay datos sin ACK → bufferea hasta el ACK; si no hay nada en vuelo manda ya (telnet: 1 char = 162B/4 segmentos). **Silly window** (Rx lee de a 1B): solución de **Clark:** no anunciar ventana hasta min(MSS, ½ buffer) libre; ACK retardado. **nmap SYN scan:** SYN → SYN+ACK (abierto; el scanner manda RST) / RST (cerrado); ISN y puertos origen varían. **Monitor sin estado:** usar el **SYN+ACK** (único con ambos flags, lo manda el server → identifica roles, prueba que el SYN fue aceptado; SYN solo puede ser scan; el ACK es indistinguible de otros). Port knocking fallido: SYN retransmitidos con TO duplicándose hasta abandonar.

⚠ **Trampas carila 2:** longest prefix, no el primer match · next hop IP → re-lookup para la interfaz · router sin ruta ni default descarta (no floodea) · ARP del next hop, no del destino remoto · la PC necesita gateway en SU subred · verificar ida y vuelta · hosts=2^{h-2} · /n: red = AND · loopback no se rutea, privadas sí se forwarden · RIP: costo recibido +1, 16=∞, sólo a vecinos · OSPF: LSP = sólo enlaces directos, a todos · SYN/FIN consumen 1 SEQ; ACK puro no · TIME_WAIT lo tiene quien cerró primero · CLOSING sólo en cierre simultáneo · quien abre (cliente) ≠ quien cierra primero · no medir RTT con retransmitidos (Karn).

G4 Ej5: 2 conexiones por 4-tupla; CONN2 #10 SYN+ACK+URG con ack=64334 (≈54322) → RST (#13). Ej7: 6/7 misma seq → retransmisión = no sirve para RTT; sirven 1→2, 2→3, 4→5, 9→10, 11→12; desde #9 server FIN_WAIT_1→FIN_WAIT_2→TIME_WAIT, cliente CLOSE_WAIT→LAST_ACK→CLOSED. Ej9 (tras 5 min): LISTEN 7777, nueva 7777→247:27351 ESTAB, 48600 ESTAB, 34688 TIME_WAIT desaparece (→2MSL). 45911 CLOSE_WAIT (recibió FIN y ackéo). Ej10 RST falso a server: server→CLOSED, cliente sigue ESTAB. Ej11: 35336 SYN_SENT→(SYN+ACK/ACK) ESTAB→(FIN/ACK) CLOSE_WAIT; 8080: SYN,SYN+ACK,ACK; t1 ≥ to+2MSL. Ej12 (≤8 paq., ambos CLOSING): SYN · SYN+ACK · ACK+HOLA · ACK+HOLA · FIN y FIN cruzados · ACK · ACK.